

垃圾短信识别-实训报告

2312966 林晖鹏

一、问题重述

题目背景：

垃圾短信 (Spam Messages, SM) 是指未经过用户同意向用户发送不愿接收的商业广告或者不符合法律规范的短信。

大数据时代的到来使得大量个人信息数据得以沉淀和积累，但是庞大的数据量缺乏有效的整理规范。

在面对巨大的短信数据时，为了保证更良好的用户体验，如何从数据中挖掘出更多有意义的信息为人们免受垃圾短信骚扰成为当前亟待解决的问题。

作业要求：

- 任务提供包括数据读取、基础模型、模型训练等基本代码。
- 参赛选手需完成核心模型构建代码，并尽可能将模型调到最佳状态。
- 模型单次推理时间不超过 10 秒。（这里不是训练时间）

题目解读：

就是通过对数据文本向量化、归一化等操作，然后选择合适的模型，进行参数调整，最终训练出一个识别垃圾短信的模型。

二、设计思想

1. 停用词

停用词是指在信息检索中，为节省存储空间和提高搜索效率，在处理自然语言数据（或文本）之前或之后会自动过滤掉某些字或词，这些字或词即被称为 Stop Words（停用词）。原题中采用的是四川大学机器学习实验室停用词库。

原本想着停用词越全越好，于是我在<https://github.com/CharyHong/Stopwords>中找到了更多的中文停用词，然后取了个并集，来帮助模型进行文本向量化。

However，加了更多的停用词之后，在同一模型下的模型正确率还不如原本的，所以在后面的实验中，我们还是使用原本提供的停用词。

可能是在新的停用词中夹杂了很多垃圾短信特有的词，导致对垃圾短信的识别能力下降。

2. 文本向量化

原文件中提供了两个文本向量化的函数，`CountVectorizer` 和 `TfidfVectorizer`，在尝试之后，发现后者的对模型性能提升的效果更好。

而对 `TfidfVectorizer` 函数的参数进行调整：

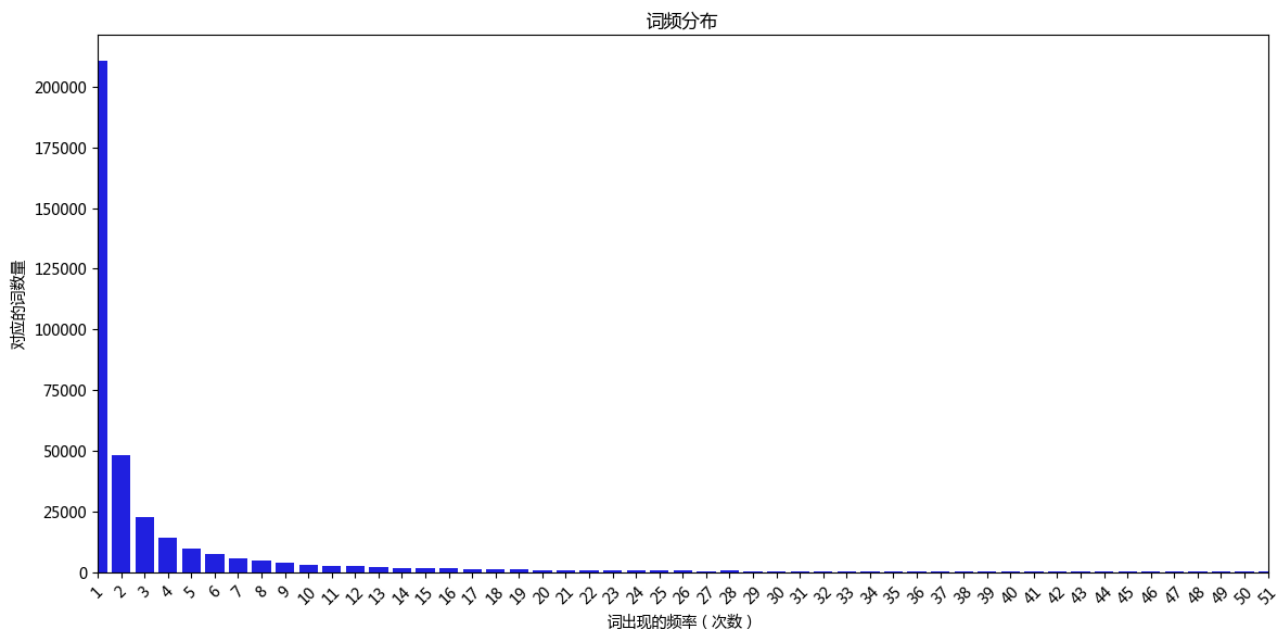
代码块

```
1  ('cv', TfidfVectorizer(  
2      token_pattern=r"(?u)\b\w+\b",  
3      stop_words=stopwords,  
4      min_df=2,                # 忽略低频词  
5      max_df=0.85,            # 忽略高频词  
6      use_idf=True,  
7      sublinear_tf=False  
8  )),
```

2.1 忽略低频词和高频词

我们尝试对每个词频的词量进行数据可视化。

发现，只出现1次的词相当多，这些低频词会导致文本向量化的结果为高维数据，导致模型训练的时候速度很慢，其次，这里面即使包含了能一针见血判断垃圾短信的词汇，模型也难以注意到这个只出现1次的词汇，所以这里我们选择直接忽略。甚至在后面调参中，把最低频次调到了5。



2.2 正则化

虽然只有在另一个文本向量化函数里面提到要增加正则化参数，但是我们这里尝试之后发现，加了这个正则化之后模型的效果更好。

2.3 use_idf 和 sublinear_tf

use_idf：启用 IDF 计数。意味着在计算单词的 TF-IDF 值时，会将单词在语料库中出现的频率与其在文档中出现的频率相结合，从而降低常见但信息量少的词语的权重。

sublinear_tf：用子线性变换。这意味着使用对数变换来改变词频值。

3. 向量降维

一开始，我们打算使用复杂一点的模型，但是这样计算效率太低了，数据向量是稀疏向量，于是我们尝试用有监督下的降维，希望提取更稠密的数据输入模型进行训练。

代码块

```
1 ('feature_selection', SelectKBest(chi2, k=1000)), # 选择前 1000 个重要特征
```

在处理后，在后续的随机森林这些模型的训练时间大大缩减，但是模型性能也有一定下降，所以后面舍弃了。

4. 向量归一化

文档中推荐了两个归一化函数 **StandardScaler** 和 **MaxAbsScaler**。

前者会导致有负向量的出现，这里我们选择使用 `MaxAbsScaler`。

5. 模型选择

5.1 前期尝试

首先，在多次尝试之后，我大胆尝试，发现网站上面的所有测试案例都是垃圾短信。

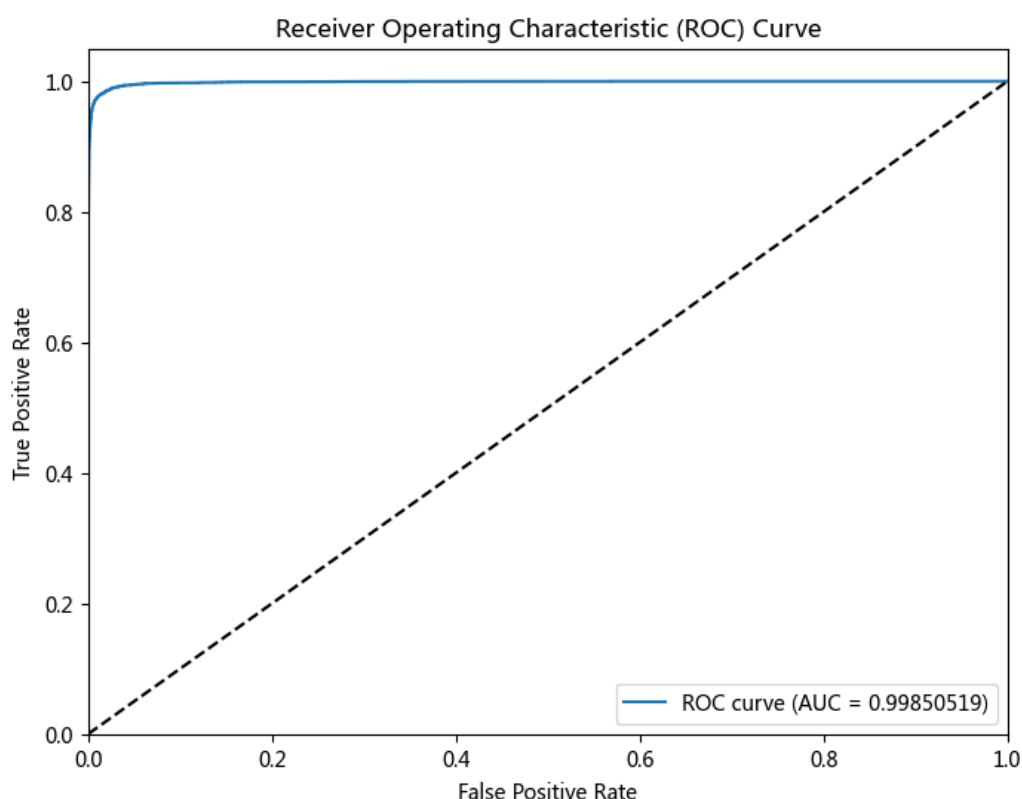
在尝试过程中，我尝试了很多模型，并调试了很多次参数，除了原本给的三个，有决策树 `DecisionTreeClassifier`、支持向量机 `SVC`、逻辑回归 `LogisticRegression`、线性支持向量机 `LinearSVC`、随机森林 `RandomForestClassifier` 还有一些多分类器增强算法：`GradientBoostingClassifier`、`AdaBoostClassifier`。

但是上面的模型，要不就是训练时间十分久，我用网站上面提供的GPU去训练都要20多分钟，对调试参数十分不方便，于是舍弃；要不就是，虽然f1分数相当高，都可以达到98甚至99了，虽然对正常短信识别能力很好，导致把f1分数拉高了，但是对垃圾短信识别的能力很差，测试结果永远都无法到达10/10。

5.2 ROC曲线

在模型选择上，此时我是很迷茫的，f1分数高的还通过不了案例，反而f1分数低的模型能通过。然后我尝试用ROC曲线和AUC值来判断选什么曲线，但是发现，由于样本及其不均衡，而且大部分模型都在正常短信的表现上较好，曲线基本长的一样，并且AUC值都接近于1。

下面是我最终模型的ROC曲线：



5.3 反思

我先是大胆尝试，全部输出识别为垃圾短信，发现10/10通过了。

在一番无论如何都无法达到10/10的折腾之后，我回过头来反思数据集。我认为是数据集一方面，垃圾短信的样本太少了，数据集的样本不平衡（然而现实生活中采集的数据集也是这样不平衡的）；另一方面，在垃圾短信样本里面，和测试数据同类型的垃圾短信估计是很少（说到底还是数据集太少，不够全面），导致模型对测试案例里面的垃圾短信无法准确识别出来。

在反思之后，我认识到，我们在这个题目背景下，不应该关注f1分数，而是要关注垃圾短信样本的召回率 `recall`，我们**宁可杀错一万，也不能放过一千**。这样，我们才有可能通过所有案例。

5.4 最终选择

最终我们选择返璞归真，回到最初给的三个模型，使用最适合稀疏数据的 `ComplementNB`。并尝试各种参数，最终调参为：

代码块

```
1 ('classifier', ComplementNB(alpha=0.1,norm=True))
```

三、代码内容

可以看到这里导入库的地方，我们跟养蛊一样，堆叠了很多模型.....

代码块

```
1 # ----- 导入相关的库 -----
2 from sklearn.pipeline import Pipeline
3 from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
4 from sklearn.naive_bayes import BernoulliNB
5 from sklearn.naive_bayes import MultinomialNB
6 from sklearn.naive_bayes import ComplementNB
7 from sklearn.svm import SVC
8 from sklearn.tree import DecisionTreeClassifier
9
10 from sklearn.preprocessing import StandardScaler,MaxAbsScaler
11 from sklearn.linear_model import LogisticRegression
12
13 from sklearn.ensemble import AdaBoostClassifier
14 from sklearn.svm import LinearSVC
15 from sklearn.ensemble import RandomForestClassifier
```

```

16 from sklearn.ensemble import GradientBoostingClassifier
17 from sklearn.feature_selection import SelectKBest, chi2
18
19 def add_stopwords(stopwords_path, stopwords):
20     with open(stopwords_path, 'r', encoding='utf-8') as f:
21         new_stopwords = f.read().splitlines()
22
23         # 将未出现过的停用词加入到现有的停用词列表中
24         updated_stopwords = list(set(stopwords) | set(new_stopwords))
25         return updated_stopwords
26
27 # 停用词库路径
28 paths = [r'scu_stopwords.txt']
29
30 # paths = [r'scu_stopwords.txt',
31 #          r'Stopwords-main/stopwords_baidu.txt',
32 #          r'Stopwords-main/stopwords_english.txt',
33 #          r'Stopwords-main/stopwords_cn.txt',
34 #          r'Stopwords-main/stopwords_full.txt',
35 #          r'Stopwords-main/stopwords_hit.txt',
36 #          r'Stopwords-main/stopwords_scu.txt',]
37 stopwords = []
38 # 读取停用词
39 for path in paths:
40     stopwords = add_stopwords(path, stopwords)
41
42
43 # pipeline_list用于传给Pipeline作为参数
44 pipeline_list = [
45     # ----- 需要完成的代码 -----
46
47     # ===== 以下代码仅供参考 =====
48     # 删除停用词，转换成向量
49     # ('cv', CountVectorizer(token_pattern=r"(?u)\b\w+\b", min_df=2,
stop_words=stopwords)),
50     ('cv', TfidfVectorizer(
51         token_pattern=r"(?u)\b\w+\b",
52         stop_words=stopwords,
53         min_df=5, # 忽略低频词
54         max_df=0.85, # 忽略高频词
55         use_idf=True,
56         sublinear_tf=False
57     )),
58
59     # ('feature_selection', SelectKBest(chi2, k=1000)), # 选择前 1000 个重要特征
60
61     # 向量归一化

```

```

62     # ('scaler', StandardScaler(with_mean=False)),
63
64     # ('feature_selection', SelectKBest(chi2, k=1000)),
65     ('scaler', MaxAbsScaler()),
66
67     # 模型设计
68     # ('classifier', MultinomialNB())
69     # ('classifier', BernoulliNB())
70     ('classifier', ComplementNB(alpha=0.1,
71                                   norm=True))
72
73     # SVC 训练太久了
74     # ('classifier', SVC(kernel='linear', C=1.0, random_state=42))
75
76     # 决策树 too long
77     # ('classifier', DecisionTreeClassifier(random_state=42))
78
79     # ('classifier', LogisticRegression(
80     # C=12.5,
81     # solver='saga', # 'liblinear'是小数据集的快速求解器
82     # max_iter=1000,
83     # random_state=42
84     # ))
85
86     # 使用 LinearSVC 作为分类器
87     # ('classifier', LinearSVC())

```

四、实验结果

系统测试

main.py

results

scu_stopwords.txt

datasets

接口测试

接口测试通过。

用例测试

测试点	状态	时长	结果
测试读取停用词库函数结果	✓	2s	read_stopwords 函数返回的类型正确
测试模型预测结果	✓	2s	通过测试，训练的分类器具备检测恶意短信的能力，分类正确比例:10/10

确认

代码块

```
1 在测试集上的混淆矩阵：
2  [[63370  7444]
3   [   12  7835]]
4  在测试集上的分类结果报告：
5           precision    recall  f1-score   support
6
7         0           1.00      0.89      0.94      70814
8         1           0.51      1.00      0.68       7847
9
10        accuracy                0.91      78661
11        macro avg           0.76      0.95      0.81      78661
12        weighted avg        0.95      0.91      0.92      78661
13
14 在测试集上的 f1-score :
15  0.6775923203320938
```

可以看到，本着追求1的召回率的原则下，1的召回率接近基本等于1了，而测试集中，把垃圾短信误判为正常短信的只有12个样本。

总的来说，这个模型对垃圾短信是“零容忍”的，即使会对正常短信错判。

五、总结

- 在本次实验中，我熟悉了关于sklearn这个机器学习库的基本使用，也查阅了这个库的api文档，对这个库的学习路径十分熟悉了。
- 其次，我感受了各种模型调试参数的痛苦，做了很多尝试，并且加深了数据集对模型训练的重要性的认知！
- 还学习了如何分析模型的好坏的各种相关指标以及ROC曲线，收获满满。